

Lab0: GitLab 实验报告

李长川 25800180107

2026 年 9 月 30 日

1 任务 1：问题回答

Q1. 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有过。课程作业和竞赛集训里和同学一起写过小程序。那时候靠微信群分工，代码互相发文件或者丢网盘，文件名经常变成 `main_new.cpp`、`final_final.cpp` 这种。合并时手动对比两份代码，改错了想回退只能重新要旧版本，也不知道是谁在什么时候改了哪一行。这次 Lab0 算是第一次正经用上版本控制。

Q2. Git 为什么要设计“暂存-提交”两个步骤？

主要是为了让一次提交在语义上完整。工作区里经常同时堆着好几件不相关的改动，有了暂存区就可以只把属于同一逻辑的改动 `git add` 进去，再写一句准确的 commit message。提交前还能用 `git status` / `git diff --cached` 检查，加错了文件可以 `git restore --staged` 撤回来。另外用 `git add -p` 可以把同一文件里的不同 hunk 拆开分别提交，历史会干净很多。

Q3. `git branch` 和 `git branch -a` 的区别是什么？

`git branch` 只列出本地分支；`git branch -a` (all) 会同时列出远程跟踪分支，即 `remotes/origin/...` 那些。后者显示的远程分支是上一次 `git fetch` / `git pull` 时的快照，不是实时的；如果远程删了某个分支，本地还要 `git fetch --prune` 清理一下过期的远程跟踪分支。

2 任务 2：模板建仓与 `main.c`

在课程给出的模板仓库 ICS-26Fall-FDU/GitLab 上点 Use this template，创建个人仓库 ChangchuanLi25800180107，然后在 WSL 里 `git clone` 到本地。

打开 `main.c`，把 TODO 那一行的 `printf` 改成 `printf("What can I say\n");`，保存后 `git add` / `git commit -m "Update main.c"` / `git push`。

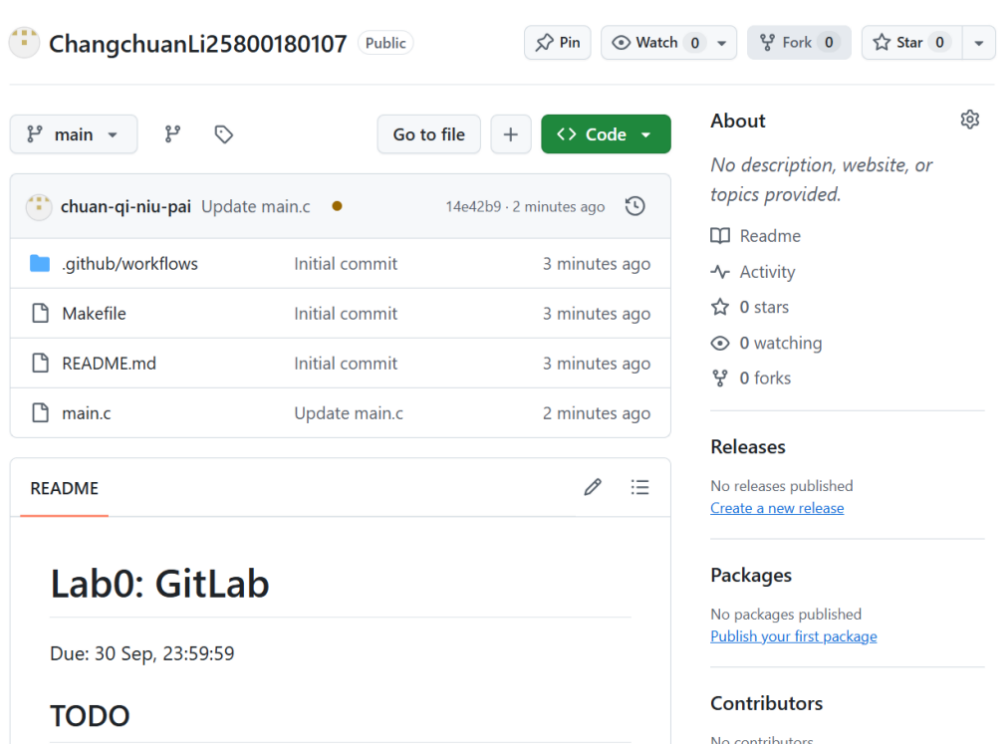


图 1: 个人仓库主页

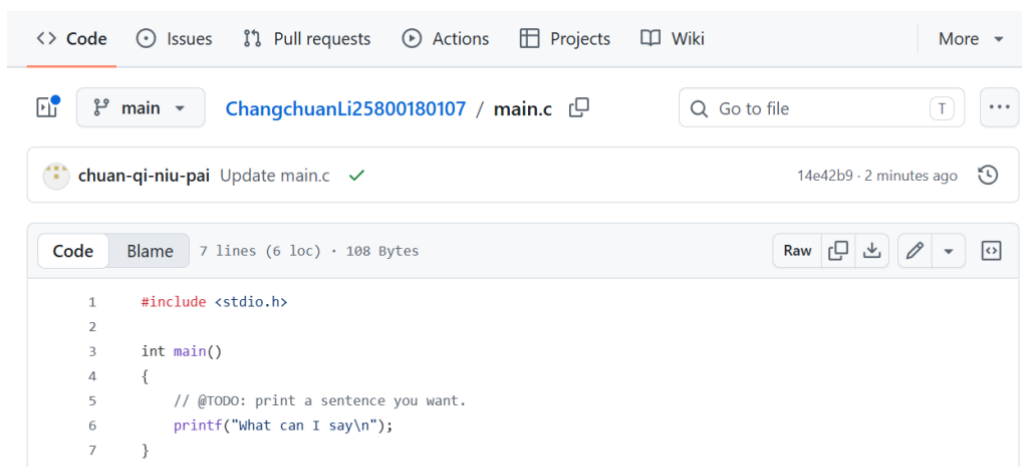


图 2: GitHub 上 main.c 的内容

3 任务 3: 阅读概括与思考

我读了其中两篇: Conventional Commits (约定式提交) 和 Semantic Versioning (语义化版本)。这两篇其实是配套的——前者管”每次提交怎么写”, 后者管”版本号怎么升”, 合在一起就是一套自动化发布的基础。

Conventional Commits. 它规定 commit message 写成

```
<type>(<scope>): <subject>
```

```
[optional body]
```

```
[optional footer(s)]
```

的格式。`type` 用 `feat` 表示新功能、`fix` 表示修 bug、`docs` 改文档、`style` 调格式、`refactor` 重构、`test` 加测试、`chore` 搞构建或依赖；`scope` 可选，标注改动影响哪个模块；`subject` 用祈使句、一句话概括。正文和 footer 里可以写清楚“为什么这么改”，如果有不兼容的 API 改动，在 footer 里写 **BREAKING CHANGE**：开头。

这样做的直接好处是：别人扫一眼 `git log` 就知道这次提交干了什么，而不是看到一堆 `"update"`、`"fix bug"`、`"aaa"`。更重要的是工具能读懂它——比如 `standard-version`、`release-please` 这类工具会自动从 `feat`：升 MINOR、从 `fix`：升 PATCH、从 **BREAKING CHANGE** 升 MAJOR，自动生成 changelog 并打 tag，整个发版流程不需要人手写版本号。

Semantic Versioning. 它要求版本号必须是 `MAJOR.MINOR.PATCH` 三段：

- MAJOR：做了不兼容的 API 修改时 +1，比如改了函数签名、删了接口；
- MINOR：以向下兼容的方式新增功能时 +1，比如多加了一个函数但旧用法还能用；
- PATCH：只做向下兼容的问题修复时 +1。

还约定了几个细节：`0.x` 初始开发阶段任何改动都可能不稳定，不承诺兼容性；预发布版本用 `1.0.0-alpha.1` 这种后缀；一旦发布就不能改内容，要改只能发新版本号。

它解决的问题是“版本号到底代表什么”。以前看到 `2.4.1 -> 3.0.0` 心里没底，不知道要不要改自己的代码；现在一看就清楚：主版本号变了就要小心，次版本号变了可以放心升，修订号变了只是修 bug。`npm`、`pip`、`cargo` 这些包管理器也靠这个判断能不能自动升级，避免“升个小版本结果程序崩了”。

为什么要学 Git. 以前我对 Git 的理解停留在“改错了能退回去”，读完这两篇才意识到，Git 真正的价值不只是版本存档，而是一套人和人、人和机器之间的协作语言。`commit message` 规范、分支模型、语义化版本这些约定，让提交历史不只是硬盘上的快照，而是可读、可被自动化工具消费的工程记录——CI/CD、自动测试、自动发版、code review 全都建立在这个基础上。

对个人来说最直观的感受是：分支让“试错”变便宜了。以前写代码我是不敢随便大改的，因为改坏了回不去；现在可以开一条 `feature/xxx` 分支随便折腾，搞砸了 `git switch main` 就回来了，`main` 永远是能跑的状态。这一学期后面几个 Lab 估计都要在 Linux 环境里写 C 代码，这种“随时能实验、随时能回退”的工作流应该会经常用到。

4 任务 4：分支管理与冲突解决

操作步骤.

1. 确认在 `main` 分支、工作区干净：`git status`。

2. 新建并切到 feature 分支: `git switch -c feature`。
3. 在 feature 分支上把 main.c 第 6 行改成 `printf("Hello from feature branch\n");` , 提交:
`git add main.c && git commit -m "feat: change message on feature branch"`。
4. 切回 main: `git switch main`, 把同一行改成 `printf("Hello from main branch\n");` , 提交。
5. `git merge feature`, 因为两个分支都改了同一行, 触发冲突。
6. 手动编辑 main.c, 删掉 `<<<<<< / ===== / >>>>>>` 标记, 保留 feature 分支的版本。
7. `git add main.c && git commit` 完成 merge commit, 最后 `git push origin main` 和 `git push origin feature`。

冲突现场. `git merge` 报 CONFLICT (content): Merge conflict in main.c, 文件里出现冲突标记:

```
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git merge fea
ture
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   main.c

no changes added to commit (use "git add" and/or "git commit -a")
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ cat main.c
#include <stdio.h>

int main()
{
    // @TODO: print a sentence you want.
    <<<<<< HEAD
        printf("Hello from main branch\n");
    =====
        printf("Hello from feature branch\n");
    >>>>>> feature
}
```

图 3: 终端报 CONFLICT, cat main.c 显示冲突标记

```
C main.c X
ChangchuanLi25800180107 > C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      <<<<<< HEAD (Current Change)
7      printf("Hello from main branch\n");
8      =====
9      printf("Hello from feature branch\n");
10     >>>>>> feature (Incoming Change)
11 }
12
```

图 4: VSCode 中的冲突视图 (红绿块)

解决冲突. 删掉三对标记, 保留 `printf("Hello from feature branch\n");`:

```
ChangchuanLi25800180107 > C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6
7      printf("Hello from feature branch\n");
8
9  }
10
```

图 5: 解决冲突后的 main.c

合并结果. 本地 `git log --graph` 显示两条分支分叉再汇合:

```
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git commit
[main 39bbf7c] Merge branch 'feature'
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git log --pretty=oneline --graph
*   39bbf7ceaac98f28cfa2481c7e13b891f9e7beac (HEAD -> main) Merge branch 'feature'
| \
| * b6bdc40786b756a2a91dd3a3e93a756691f58d0a (feature) feat: change message on feature branch
| |
| * 1be467baae9de926d561fceb30b77817bf671a45 feat: change message on main branch
|/
* 14e42b9f1e4dfcd69d82984ba405f1e876073741 (origin/main, origin/HEAD) Update main.c
* 297fb2887b8692952c2ea512b3519bf604092e2b Initial commit
```

图 6: git log -graph 输出

推到 GitHub 后, Network 页面也能看到分叉合并的图形:

Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.

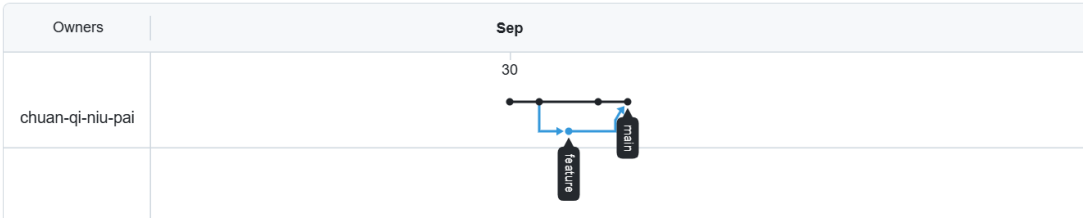


图 7: GitHub Network graph

```
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git push orig
in main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 32 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 842 bytes | 76.00 KiB/s, done.
Total 9 (delta 6), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (6/6), completed with 1 local object.
To github.com:chuan-qi-niu-pai/ChangchuanLi25800180107.git
  14e42b9..39bbf7c  main -> main
stik@Stik:/mnt/c/Users/chuan/Desktop/text/ChangchuanLi25800180107$ git push orig
in feature
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/chuan-qi-niu-pai/ChangchuanLi25800180107/pull/new/feature
remote:
To github.com:chuan-qi-niu-pai/ChangchuanLi25800180107.git
 * [new branch]      feature -> feature
```

图 8: push 到远程

5 仓库链接

<https://github.com/chuan-qi-niu-pai/ChangchuanLi25800180107>